1) What is JPA?

Interview Question

What is JPA in Java and why is it used in Spring Boot applications?

Detailed Answer

JPA stands for **Java Persistence API**. It is a **Java specification** used for managing relational data in Java applications.

It provides a standard way to:

- Map Java classes to database tables
- Perform CRUD operations
- Handle relationships between entities
- Manage persistence lifecycle

Important point:

JPA is **not an implementation**.
It only defines interfaces and rules.

To actually use JPA, we need an implementation such as:

- **Hibernate** (most common)
- EclipseLink
- OpenJPA

In Spring Boot, JPA is commonly used with **Spring Data JPA**, which reduces boilerplate code for database access.

Why use JPA?

Without JPA, developers need to:

- Write JDBC code manually
- Manage SQL statements
- Handle ResultSet mapping
- Manage connections and transactions more manually

With JPA:

- Less boilerplate
- Object-oriented approach
- Easier CRUD
- Better maintainability

Simple Explanation

JPA means:

“A standard Java way to map Java objects to database tables and perform DB operations.”

Example

```
@Entity
public class User {

    @Id
    private Long id;

    private String name;
}
```

Here:

- `User` = Java class
- JPA maps it to a DB table

Interviewer Cross-Question

Q: Is JPA a framework or specification?
A: JPA is a **specification**, not a framework or direct implementation.

2) What is Hibernate?

Interview Question

What is Hibernate and how is it related to JPA?

Detailed Answer

Hibernate is an **ORM framework** and one of the most popular **implementations of JPA**.

ORM = **Object Relational Mapping**

Hibernate allows Java objects to be mapped to relational database tables and provides:

- CRUD operations
- Query support (HQL/JPQL)
- Caching
- Lazy loading
- Relationship management
- Dirty checking
- Transaction integration

Relationship with JPA:

- JPA = specification
- Hibernate = implementation/provider

In most Spring Boot projects:

- We use JPA annotations (`@Entity`, `@Id`, etc.)
- Hibernate works behind the scenes to execute SQL

Example:

When we call:

```
userRepository.save(user);
```

Spring Data JPA delegates to Hibernate internally, and Hibernate generates the SQL.

Simple Explanation

Hibernate means:

"The engine that actually implements JPA and talks to the database."

Example

```
spring.jpa.hibernate.ddl-auto=update
```

This property tells Hibernate how to manage schema changes.

Interviewer Cross-Question

Q: Can we use JPA without Hibernate?
A: Yes. JPA can work with other providers like EclipseLink, but Hibernate is the most common.

3) Difference between JPA and Hibernate

Interview Question

What is the difference between JPA and Hibernate?

Detailed Answer

This is a very common interview question.

JPA

- A **specification**
- Defines standard interfaces and annotations
- Example:
  - `EntityManager`
  - `@Entity`
  - `@Id`

Hibernate

- An **implementation** of JPA
- Provides the actual logic
- Executes SQL
- Manages persistence context
- Also provides extra features beyond JPA

Key point:

We usually write code using **JPA standard APIs**, so our code is more portable.
Hibernate is used underneath to execute it.

Interview-safe answer:

- **JPA tells what should be done**
- **Hibernate does it**

Simple Explanation

- **JPA** = rules / contract
- **Hibernate** = actual engine / implementation

Example

```
@Entity
public class Employee {
    @Id
    private Long id;
}
```

`@Entity` is a **JPA annotation**, but Hibernate interprets it and performs the mapping.

Interviewer Cross-Question

Q: Why prefer JPA APIs over Hibernate-specific APIs?
A: To keep the application loosely coupled and portable across providers.

4) What is Spring Data JPA?

Interview Question

What is Spring Data JPA and how does it simplify database access?

Detailed Answer

Spring Data JPA is a Spring module built on top of JPA that simplifies database access by reducing boilerplate code.

Without Spring Data JPA, we often need to:

- Create DAO classes manually
- Inject `EntityManager`
- Write repetitive CRUD code

With Spring Data JPA:

- We define repository interfaces
- Extend `JpaRepository`
- Spring automatically provides implementation at runtime

Benefits:

- Built-in CRUD methods
- Derived query methods (`findByName`)
- Pagination and sorting
- JPQL and native query support
- Integration with transactions
- Cleaner architecture

Example:

```
public interface UserRepository extends JpaRepository<User,
Long> {
}
```

No implementation class needed.

Simple Explanation

Spring Data JPA means:

“Spring makes JPA much easier by generating repository code automatically.”

Example

```
@Repository
```

```
public interface UserRepository extends JpaRepository<User,
Long> {
}
```

Now we automatically get:

- `save()`
- `findById()`
- `findAll()`
- `deleteById()`

Interviewer Cross-Question

Q: Do we need to implement `JpaRepository` methods manually?
A: No, Spring generates the implementation automatically at runtime.

5) What is `@Entity`?

Interview Question

What is `@Entity` in JPA and why is it required?

Detailed Answer

`@Entity` is a JPA annotation used to mark a Java class as a **persistent entity**.

It tells JPA/Hibernate:

- This class should be mapped to a database table
- Each object instance represents a row in the table

Rules for an entity class:

- Must be annotated with `@Entity`
- Must have a no-arg constructor (at least protected/public)
- Must have a primary key (`@Id`)
- Should not be final in most cases (Hibernate proxies can be affected)
- Usually placed in entity/domain package

Default behavior:

If `@Table` is not provided:

- Table name defaults to class name (or naming strategy-based conversion)

Simple Explanation

`@Entity` means:

“This Java class is a database table representation.”

Example

```
@Entity
public class Product {

    @Id
    private Long id;

    private String name;
    private Double price;
}
```

Interviewer Cross-Question

Q: Can a class be used with JPA without `@Entity`?
A: No, JPA won’t treat it as a persistent class without `@Entity` (or equivalent mapping configuration).

6) What is `@Id`?

Interview Question

What is `@Id` in JPA and why is it important?

Detailed Answer

`@Id` is used to mark the **primary key** field of an entity.

Every JPA entity must have a unique identifier because JPA uses it to:

- Identify records uniquely
- Track entity state in persistence context
- Perform updates and deletes correctly
- Manage caching and dirty checking

Why it is important:

Without `@Id`, JPA cannot determine:

- Which row belongs to which entity
- Whether to insert or update
- How to uniquely manage the object

Primary key can be:

- Simple key (single field)
- Composite key (using `@EmbeddedId` or `@IdClass` — advanced topic later)

Simple Explanation

`@Id` means:

"This field is the unique identifier (primary key) of the entity."

Example

```
@Entity
public class Student {

    @Id
    private Long id;

    private String name;
}
```

Interviewer Cross-Question

Q: Can JPA entity exist without `@Id`?
A: Practically no. Every entity must define an identifier.

7) What is `@GeneratedValue`?

Interview Question

What is `@GeneratedValue` in JPA and what are its common strategies?

Detailed Answer

`@GeneratedValue` is used with `@Id` to specify how the primary key value should be generated automatically.

This avoids manually setting IDs for every new entity.

Common strategies:

1. **AUTO**

    - JPA chooses the best strategy based on DB/provider
2. **IDENTITY**
    - DB auto-increment column
    - Common in MySQL
3. **SEQUENCE**
    - Uses database sequence
    - Common in Oracle/PostgreSQL
4. **TABLE**
    - Uses a separate table to generate IDs
    - Less commonly used

Important interview note:

- `IDENTITY` is simple but can affect batching in some cases
- `SEQUENCE` is often preferred in databases that support it

Simple Explanation

`@GeneratedValue` means:

"Let the system/database generate the primary key automatically."

Example

```
@Entity
public class OrderEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productName;
}
```

Interviewer Cross-Question

Q: Which strategy is commonly used with MySQL?
A: `GenerationType.IDENTITY`

8) What is `JpaRepository`?

Interview Question

What is `JpaRepository` and why is it commonly used in Spring Data JPA?

Detailed Answer

`JpaRepository` is an interface provided by Spring Data JPA that offers a rich set of built-in methods for database operations.

It extends:

- `PagingAndSortingRepository`
- which extends `CrudRepository`

Features of `JpaRepository`:

- CRUD operations
- Pagination
- Sorting
- Batch operations
- Flush-related methods

Common methods:

- `save()`
- `findById()`
- `findAll()`
- `deleteById()`
- `findAll(Pageable pageable)`
- `flush()`

Why use it?

It eliminates boilerplate DAO code and makes repositories concise and powerful.

Simple Explanation

`JpaRepository` means:

“A ready-made repository interface with CRUD + pagination + sorting support.”

Example

```
public interface EmployeeRepository extends
JpaRepository<Employee, Long> {
}
```

Now Spring gives implementation automatically.

Interviewer Cross-Question

Q: What are the two generic parameters in `JpaRepository<Employee, Long>`?
A:

- `Employee` = entity type
- `Long` = primary key type

9) Difference between `CrudRepository` and `JpaRepository`

Interview Question

What is the difference between `CrudRepository` and `JpaRepository`?

Detailed Answer

Both are repository interfaces in Spring Data, but `JpaRepository` is more feature-rich.

`CrudRepository`

Provides basic CRUD operations:

- `save()`
- `findById()`
- `findAll()`
- `deleteById()`

`JpaRepository`

Extends `PagingAndSortingRepository` and indirectly `CrudRepository`, so it includes:

- All CRUD methods
- Pagination support
- Sorting support
- JPA-specific features like:
  - `flush()`
  - `saveAndFlush()`

- batch deletes

Practical recommendation:

In most Spring Boot JPA projects, we directly use:

- `JpaRepository`

Because it gives more capabilities and is the common standard.

Simple Explanation

- `CrudRepository` = basic CRUD only
- `JpaRepository` = CRUD + paging + sorting + extra JPA features

Example

```
public interface UserRepository extends CrudRepository<User,
Long> {
}
```

```
public interface UserRepository extends JpaRepository<User,
Long> {
}
```

Second one is more commonly used.

Interviewer Cross-Question

Q: Which one do you prefer in real projects?
A: Usually `JpaRepository`.

10) Explain common `JpaRepository` methods: `save()`, `findById()`, `findAll()`, `deleteById()`

Interview Question

What are the most commonly used methods in `JpaRepository`, and how do they work?

Detailed Answer

These are the most commonly used built-in methods:

1. `save(entity)`

Used to insert or update an entity.

- If entity is new (no existing ID / not managed) → INSERT
- If entity already exists / managed → UPDATE (depending on context)

2. `findById(id)`

Fetches an entity by primary key.

- Returns `Optional<T>`
- Helps avoid null handling

3. `findAll()`

Returns all records of that entity type.

- Returns `List<T>`

4. `deleteById(id)`

Deletes a record by primary key.

- If entity not found, behavior depends on context/provider method path

Why these matter?

These methods cover most basic CRUD operations without writing custom SQL or JPQL.

Simple Explanation

These 4 methods are your everyday JPA tools:

- `save()` → insert/update
- `findById()` → get one
- `findAll()` → get all
- `deleteById()` → delete one

Example

```
@Service
```

```
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public User createUser(User user) {
        return userRepository.save(user);
    }

    public User getUser(Long id) {
        return userRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("User
not found"));
    }

    public List<User> getAllUsers() {
        return userRepository.findAll();
    }

    public void deleteUser(Long id) {
        userRepository.deleteById(id);
    }
}
```

11) What are derived query methods in Spring Data JPA?

Interview Question

What are derived query methods in Spring Data JPA and how do they work?

Detailed Answer

Derived query methods are query methods where Spring Data JPA automatically generates the query **based on the method name**.

This means we don't need to write JPQL or SQL manually for many common queries.

Example:

```
findByName(String name)
```

Spring interprets this and generates a query like:

```
select * from user where name = ?
```

Common patterns:

- `findByName`
- `findByEmail`
- `findByStatus`
- `findByNameAndEmail`
- `findByAgeGreaterThan`
- `findByNameContaining`
- `findByOrderByNameAsc`

Benefits:

- Less boilerplate
- Readable method names
- Fast development for simple queries

Limitations:

- Method names can become too long/complex
- Not ideal for very complex queries

Simple Explanation

Derived query method means:

"Spring creates the query automatically from the repository method name."

Example

```
public interface UserRepository extends JpaRepository<User,
Long> {

    List<User> findByName(String name);

    List<User> findByEmailAndStatus(String email, String
status);

    List<User> findByAgeGreaterThan(int age);
}
```

Interviewer Cross-Question

Q: When should we avoid derived query methods?
A: When the query becomes too complex or the method name becomes too long and unreadable.

12) Why does `findById()` return `Optional<T>`?

Interview Question

Why does `findById()` return `Optional<T>` in Spring Data JPA?

Detailed Answer

`findById()` returns `Optional<T>` because the entity may or may not exist in the database.

This is better than returning `null` because:

- It forces explicit handling of missing values
- Reduces `NullPointerException`
- Makes the code more expressive
- Encourages clean patterns like `orElseThrow()`

Typical REST usage:

```
userRepository.findById(id).orElseThrow(...)
```

This maps nicely to:

- `404 NOT FOUND` if record doesn't exist

Simple Explanation

`Optional<T>` means:

"The record may be present or may be absent."

Example

```
public User getUser(Long id) {
    return userRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("User not
found"));
}
```

Interviewer Cross-Question

Q: Why is `orElseThrow()` better than `isPresent()` + `get()`?
A: Cleaner, more concise, and more expressive.

13) What is `@Query` in Spring Data JPA?

Interview Question

What is `@Query` in Spring Data JPA and when do we use it?

Detailed Answer

`@Query` is used to define a **custom query** directly on a repository method when:

- Derived query methods are not enough
- Query logic is complex
- We want explicit control over the query

It supports:

- **JPQL** queries (default)
- **Native SQL** queries (`nativeQuery = true`)

When to use `@Query`:

- Complex filtering
- Joins
- Aggregations
- Custom update/delete
- More readable than extremely long method names

Simple Explanation

`@Query` means:

"Write your own query directly on the repository method."

Example (JPQL)

```
public interface UserRepository extends JpaRepository<User,
Long> {

    @Query("SELECT u FROM User u WHERE u.email = :email")
    Optional<User> findUserByEmail(@Param("email") String
email);
}
```

Interviewer Cross-Question

Q: Is `@Query` JPQL by default or SQL by default?
A: By default, it uses **JPQL**.

14) What is the difference between JPQL and Native Query?

Interview Question

What is the difference between JPQL and Native Query in Spring Data JPA?

Detailed Answer

This is one of the most common JPA interview questions.

JPQL (Java Persistence Query Language)

- Query is written using **entity names and entity field names**
- It is **database-independent**
- Works on object model, not directly on tables

Example:

```
SELECT u FROM User u WHERE u.name = :name
```

Here:

- `User` = entity class
- `name` = entity field

Native Query

- Query is written in **actual SQL**
- Uses **table names and column names**
- Database-specific syntax can be used

Example:

```
SELECT * FROM users WHERE user_name = :name
```

When to use JPQL?

- Preferred in most cases
- Portable
- Works with entity model

When to use Native Query?

- Complex SQL
- Vendor-specific functions
- Existing legacy SQL
- Performance tuning in some cases

Simple Explanation

- **JPQL** = query using **Java entity model**
- **Native Query** = query using **actual SQL table/column names**

Example

JPQL

```
@Query("SELECT u FROM User u WHERE u.status = :status")
List<User> findByStatus(@Param("status") String status);
```

Native SQL

```
@Query(value = "SELECT * FROM users WHERE status = :status",
nativeQuery = true)
List<User> findByStatusNative(@Param("status") String
status);
```

Interviewer Cross-Question

Q: Which one should we prefer in most projects?
A: Prefer **JPQL** unless there's a strong reason to use native SQL.

15) What is `@Modifying` and when is it required?

Interview Question

What is `@Modifying` in Spring Data JPA and when do we use it?

Detailed Answer

`@Modifying` is used with repository methods that execute **update** or **delete** queries using `@Query`.

By default, `@Query` assumes a **SELECT** query.

If the query is:

- `UPDATE`
- `DELETE`
- sometimes DDL (rare in app code)

Then we must add `@Modifying`.

Why?

Because Spring Data JPA needs to know that this query changes data instead of returning entities.

Important:

`@Modifying` is usually used together with:

- `@Transactional`

Simple Explanation

`@Modifying` means:

“This custom query changes data, it is not a select query.”

Example

```
public interface UserRepository extends JpaRepository<User,
Long> {

    @Modifying
    @Query("UPDATE User u SET u.status = :status WHERE u.id
= :id")
    int updateUserStatus(@Param("id") Long id,
@Param("status") String status);
}
```

Interviewer Cross-Question

Q: What happens if we forget `@Modifying` on an update query?
A: Spring treats it like a select query and it will fail at runtime.

16) What is `@Transactional` in JPA context?

Interview Question

Why is `@Transactional` important in Spring Data JPA?

Detailed Answer

`@Transactional` is used to define a transactional boundary around database operations.

A transaction ensures:

- All operations succeed together, or
- If something fails, everything is rolled back

In JPA context, `@Transactional` is important because:

- It manages commit/rollback
- It keeps persistence context active
- It supports lazy loading within transaction
- It is required for custom update/delete repository methods in many cases
- It ensures data consistency

Typical place to use:

- **Service layer** (best practice)

Example:

If you save order and payment:

- If payment save fails, order save should also roll back

Simple Explanation

`@Transactional` means:

"Treat these DB operations as one unit. Either all succeed or all fail."

Example

```
@Service
public class UserService {
```

```
    @Autowired
    private UserRepository userRepository;

    @Transactional
    public void updateUserStatus(Long id, String status) {
        userRepository.updateUserStatus(id, status);
    }
}
```

Interviewer Cross-Question

Q: Where should `@Transactional` usually be placed?
A: Usually in the **service layer**, not directly in controller.

17) What are `@Column`, `@Table`, and `@Transient`?

Interview Question

What are `@Column`, `@Table`, and `@Transient` annotations in JPA?

Detailed Answer

These are common JPA mapping annotations.

`@Table`

Used at class level to specify the table name.

If not provided:

- JPA/Hibernate uses default naming strategy

```
@Entity
@Table(name = "users")
public class User { }
```

`@Column`

Used at field level to customize column mapping.

It can define:

- column name
- nullable
- length
- unique
- precision/scale

```
@Column(name = "user_name", nullable = false, length = 100)
private String name;
```

`@Transient`

Marks a field that should **not be persisted** to the database.

Useful for:

- Calculated fields
- Temporary runtime values
- UI/helper data

```
@Transient
private String fullName;
```

Simple Explanation

- `@Table` = table name mapping
- `@Column` = column customization
- `@Transient` = ignore this field in DB

Example

```
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    private Long id;

    @Column(name = "emp_name", nullable = false)
    private String name;

    @Transient
    private String temporaryDisplayValue;
}
```

Interviewer Cross-Question

Q: What is the difference between Java `transient` keyword and JPA `@Transient`?
A:

- `@Transient` → ignored by JPA persistence
- `transient` keyword → ignored by Java serialization

18) What is `@Enumerated` in JPA?

Interview Question

What is `@Enumerated` in JPA and why should we be careful with `EnumType.ORDINAL`?

Detailed Answer

`@Enumerated` is used when an entity field is of type **enum**.

It tells JPA how to store enum values in the database.

Two strategies:

1. `EnumType.STRING`

Stores enum name as text.
Example:

- `ACTIVE`
- `INACTIVE`

2. `EnumType.ORDINAL`

Stores enum position as integer.
Example:

- `ACTIVE = 0`
- `INACTIVE = 1`

Best practice:

Use **`EnumType.STRING`**

Why avoid `ORDINAL`?

If enum order changes later:

```
ACTIVE, INACTIVE
```

becomes:

```
NEW, ACTIVE, INACTIVE
```

Then stored DB values become incorrect logically.

Simple Explanation

`@Enumerated` means:

"This field is an enum; tell JPA how to store it."

Example

```
public enum Status {
    ACTIVE,
    INACTIVE
}


@Entity
public class User {

    @Id
    private Long id;

    @Enumerated(EnumType.STRING)
    private Status status;
}
```

Interviewer Cross-Question

Q: Which enum storage strategy is safer in real projects?
A: `EnumType.STRING`

19) How do pagination and sorting work in Spring Data JPA?

Interview Question

How do you implement pagination and sorting in Spring Data JPA?

Detailed Answer

Pagination and sorting are built into Spring Data JPA using:

- `Pageable`
- `Page<T>`
- `Sort`

Why pagination?

When the dataset is large, returning all rows at once is inefficient.

Pagination with `Pageable`

Spring automatically creates a `Pageable` object from request parameters like:

- `page`
- `size`
- `sort`

Return type:

- `Page<T>` → includes:
  - content
  - total pages
  - total elements
  - current page
  - etc.

Sorting:

Can be applied:

- standalone using `Sort`
- or inside `Pageable`

Simple Explanation

Pagination = “fetch data in chunks”
Sorting = “fetch data in ordered form”

Example

Repository

```
public interface UserRepository extends JpaRepository<User,
Long> {
}
```

Service / Controller

```
@GetMapping("/users")
public Page<User> getUsers(Pageable pageable) {
    return userRepository.findAll(pageable);
}
```

Sample Request

```
GET /users?page=0&size=5&sort=name,asc
```

Sorting only

```
List<User> users =
userRepository.findAll(Sort.by(Sort.Direction.DESC, "name"));
```

Interviewer Cross-Question

Q: What is the difference between `Page` and `List`?
A: `Page` includes pagination metadata; `List` is just raw data.

20) What is JPA Auditing? (`createdAt`, `updatedAt`)

Interview Question

What is JPA Auditing and how do you automatically maintain `createdAt` and `updatedAt` fields?

Detailed Answer

JPA Auditing is a feature that automatically tracks metadata such as:

- When an entity was created
- When it was last updated
- Who created it
- Who updated it (advanced)

Common fields:

- `createdAt`
- `updatedAt`

Annotations used:

- `@CreatedDate`
- `@LastModifiedDate`
- `@EntityListeners(AuditingEntityListener.class)`

Required:

Enable auditing in configuration:

```
@EnableJpaAuditing
```

Benefits:

- Reduces boilerplate
- Ensures consistent audit fields
- Useful in enterprise applications

Simple Explanation

JPA Auditing means:

“Automatically fill created/updated timestamps without manually setting them every time.”

Example

Config

```
@Configuration
@EnableJpaAuditing
public class JpaConfig {
}
```

Entity

```
@Entity
@EntityListeners(AuditingEntityListener.class)
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;

    private String name;

    @CreatedDate
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;
}
```

21) What is @OneToOne in JPA?

Interview Question

What is `@OneToOne` in JPA and when do we use it?

Detailed Answer

`@OneToOne` defines a relationship where **one entity is associated with exactly one other entity**.

Real-world examples:

- User ↔ Passport
- Employee ↔ Locker
- Person ↔ Aadhaar details (conceptually)

How it works:

One table contains a foreign key referencing the other table.

Important concepts:

- One side can be the **owning side**
- The other side can be the **inverse side** using `mappedBy`

Use when:

- Relationship is truly 1-to-1
- Data is logically separate but tightly linked

Simple Explanation

`@OneToOne` means:

"One record is linked to exactly one record in another table."

Example

```
@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToOne
    @JoinColumn(name = "passport_id")
    private Passport passport;
}


@Entity
public class Passport {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String passportNumber;
}
```

Interviewer Cross-Question

Q: Which side owns the relationship in the above example?
A: `User` owns it because it contains `@JoinColumn`.

22) What is `@OneToMany` in JPA?

Interview Question

What is `@OneToMany` in JPA and how is it commonly used?

Detailed Answer

`@OneToMany` defines a relationship where **one entity is associated with multiple child entities**.

Real-world examples:

- Department → Many Employees
- Customer → Many Orders
- Blog → Many Comments

Important note:

In relational DB, the foreign key usually exists on the **many side**.

So in JPA:

- `@OneToMany` is often paired with `@ManyToOne`

Best practice:

Use **bi-directional mapping** only when needed, because:

- Serialization issues
- Complexity
- Lazy loading problems

Simple Explanation

`@OneToMany` means:

"One parent can have many children."

Example

```
@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department")
    private List<Employee> employees;
}


@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private String name;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}
```

Interviewer Cross-Question

Q: Where is the foreign key actually stored?
A: On the **many side**, in `Employee` table (`department_id`).

23) What is `@ManyToOne` in JPA?

Interview Question

What is `@ManyToOne` in JPA and why is it very common?

Detailed Answer

`@ManyToOne` defines a relationship where **many child records belong to one parent record**.

This is one of the **most common** relationships in real-world applications.

Real-world examples:

- Many employees belong to one department
- Many orders belong to one customer
- Many comments belong to one post

Why very common?

Because relational databases naturally store the foreign key on the child table.

Best practice:

Usually, `@ManyToOne` is simpler and more practical than starting from `@OneToMany`.

Simple Explanation

`@ManyToOne` means:

"Many child records point to one parent record."

Example

```
@Entity
public class OrderEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String productName;

    @ManyToOne
    @JoinColumn(name = "customer_id")
    private Customer customer;
}


@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

Interviewer Cross-Question

Q: Which relationship is more commonly used directly in JPA projects: `@OneToMany` or `@ManyToOne`?
A: `@ManyToOne` is often more common and simpler.

24) What is `@ManyToMany` in JPA?

Interview Question

What is `@ManyToMany` in JPA and when should we be careful using it?

Detailed Answer

`@ManyToMany` defines a relationship where:

- Many records of Entity A relate to many records of Entity B

Real-world examples:

- Students ↔ Courses
- Users ↔ Roles
- Products ↔ Tags

How it works in DB:

It requires a **join table**.

Important caution:

In real-world enterprise projects, direct `@ManyToMany` is often avoided when:

- Extra columns are needed in the join table
- Business logic exists on the association

In such cases, we create an explicit join entity instead.

Simple Explanation

`@ManyToMany` means:

"Many records on both sides can be linked to many records on the other side."

Example

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private List<Course> courses;
}


@Entity
```

```
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
}
```

Interviewer Cross-Question

Q: Why do many teams avoid direct `@ManyToMany` in real projects?
A: Because if the join table needs extra columns (like `assignedAt`, `status`), a separate join entity is better.

25) What is the owning side vs `mappedBy` in JPA?

Interview Question

What is the owning side in JPA relationships, and what does `mappedBy` mean?

Detailed Answer

In bi-directional JPA relationships, one side must be the **owning side**, and the other side is the **inverse (non-owning) side**.

Owning side:

- Controls the relationship in the database
- Usually contains `@JoinColumn`

Inverse side:

- Uses `mappedBy`
- Refers to the field name in the owning side
- Does **not** control the foreign key

Why important?

If you misunderstand owning side:

- Relationship updates may not persist as expected
- Foreign key may not be updated

Example:

In `Department ↔ Employee`

- `Employee` has `@ManyToOne @JoinColumn`
- `Department` has `@OneToMany(mappedBy = "department")`

So:

- `Employee` = owning side
- `Department` = inverse side

Simple Explanation

- Owning side = the side that controls DB foreign key
- `mappedBy` = "I am the opposite side; other side owns it"

Example

```
@OneToMany(mappedBy = "department")
private List<Employee> employees;
```

Here `mappedBy = "department"` means:

- The `department` field in `Employee` owns the relationship

Interviewer Cross-Question

Q: Which side should be updated for the relationship to persist correctly?
A: The **owning side** must be set correctly.

26) What are cascade types in JPA?

Interview Question

What are cascade types in JPA and when do we use them?

Detailed Answer

Cascade types define whether operations performed on a parent entity should automatically apply to related child entities.

Common cascade types:

- `PERSIST`
- `MERGE`
- `REMOVE`

- `REFRESH`
- `DETACH`
- `ALL`

Example:

If `Department` has employees:

- Saving department can also save employees
- Deleting department can also delete employees (depending on cascade)

Common usage:

```
@OneToMany(mappedBy = "department", cascade =
CascadeType.ALL)
private List<Employee> employees;
```

Important caution:

`CascadeType.REMOVE` can be dangerous if used carelessly.

Best practice:

Use cascade only when child lifecycle is truly dependent on parent.

Simple Explanation

Cascade means:

"If I save/update/delete parent, should the same happen automatically to children?"

Example

```
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "passport_id")
private Passport passport;
```

Saving `User` can also save `Passport`.

Interviewer Cross-Question

Q: Is `CascadeType.ALL` always a good idea?
A: No. Use it carefully; it may delete or update child records unintentionally.

27) What is `FetchType.LAZY` vs `FetchType.EAGER`?

Interview Question

What is the difference between `FetchType.LAZY` and `FetchType.EAGER` in JPA?

Detailed Answer

Fetch type defines **when related data is loaded**.

`FetchType.LAZY`

- Related entity is loaded **only when accessed**
- Improves performance if related data is not always needed
- Often preferred in real projects

`FetchType.EAGER`

- Related entity is loaded **immediately along with parent**
- Can lead to unnecessary queries and performance issues

Default behavior:

- `@ManyToOne` → **EAGER** by default
- `@OneToMany` → **LAZY** by default

Best practice:

Be explicit and prefer **LAZY** where possible.

Simple Explanation

- **LAZY** = load later when needed
- **EAGER** = load immediately

Example

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "department_id")
private Department department;
```

Interviewer Cross-Question

Q: Which fetch type is generally safer for performance?
A: Usually **LAZY**.

28) What is the N+1 problem in JPA?

Interview Question

What is the N+1 query problem in JPA/Hibernate?

Detailed Answer

The N+1 problem occurs when:

1. One query fetches a list of parent entities (**1 query**)
2. Then for each parent, an additional query fetches related child data (**N queries**)

So total queries = **1 + N**

Example:

- Fetch 100 orders
- Then access customer for each order
- 1 query for orders + 100 queries for customers

Why it is bad?

- Performance degradation
- Too many DB round trips
- Common hidden issue in production

Common solutions:

- `JOIN FETCH` in JPQL
- `@EntityGraph`
- Batch fetching
- DTO projections

Simple Explanation

N+1 means:

"You thought one query would be enough, but JPA silently runs many extra queries."

Example Scenario

```
List<OrderEntity> orders = orderRepository.findAll();

for (OrderEntity order : orders) {
```

```
    System.out.println(order.getCustomer().getName());
}
```

If `customer` is lazily loaded, this can trigger N+1.

Interviewer Cross-Question

Q: Is N+1 caused only by LAZY loading?
A: Most commonly associated with lazy loading, but even eager fetching can still create inefficient query patterns depending on access and mapping.

29) What is `LazyInitializationException`?

Interview Question

What is `LazyInitializationException` in Hibernate and why does it happen?

Detailed Answer

`LazyInitializationException` occurs when we try to access a **lazy-loaded association** after the Hibernate session / persistence context is already closed.

Typical scenario:

1. Entity loaded inside transaction
2. Transaction/session ends
3. Later code tries to access a lazy field
4. Hibernate cannot fetch it anymore
5. Exception occurs

Example:

```
user.getOrders().size();
```

If `orders` is lazy and accessed outside transaction, exception may occur.

Common solutions:

- Access required data inside transaction
- Use `JOIN FETCH`
- Convert to DTO inside service layer
- Use `@EntityGraph`
- Avoid Open Session in View as a “lazy fix” strategy in serious backend APIs

Simple Explanation

This exception means:

"JPA wanted to load lazy data later, but the DB session was already closed."

Example

```
@Transactional
public UserResponseDto getUser(Long id) {
    User user = userRepository.findById(id).orElseThrow();
    int orderCount = user.getOrders().size(); // safe inside
transaction
    return new UserResponseDto(user.getId(), user.getName(),
orderCount);
}
```

Interviewer Cross-Question

Q: Is changing everything to EAGER a good fix?
A: No. That can create performance issues and N+1 problems. Better to fetch intentionally.

30) What is persistence context / entity lifecycle in JPA?

Interview Question

What is the persistence context in JPA, and what are the main entity lifecycle states?

Detailed Answer

Persistence context is the set of entities currently managed by JPA/Hibernate within a session/transaction.

It acts like a **first-level cache** and tracks entity changes.

Why important?

Because JPA can:

- Detect changes automatically (dirty checking)
- Avoid duplicate DB fetches for same entity in same context
- Manage insert/update behavior

Main Entity Lifecycle States

1. Transient

- New object
- Not associated with persistence context
- Not in DB

```
User user = new User();
```

2. Persistent / Managed

- Entity is associated with persistence context
- JPA tracks changes
- Changes can be auto-synced to DB

```
User user = entityManager.find(User.class, 1L);
```

3. Detached

- Entity was managed earlier, but persistence context is closed or entity detached
- Changes are not automatically tracked

4. Removed

- Entity marked for deletion
- Will be deleted when transaction commits

Why interviewers ask this?

Because this explains:

- `save()` behavior
- dirty checking
- lazy loading
- transaction behavior

Simple Explanation

Persistence context means:

“JPA’s working memory for currently managed entities.”

Entity states:

- Transient

- Managed
- Detached
- Removed

Example

```
@Transactional
public void updateUser(Long id) {
    User user = userRepository.findById(id).orElseThrow();
    user.setName("Murali Updated");
    // No explicit save required in some managed scenarios
because of dirty checking
}
```